

lab

April 28, 2026

1 Optics Laboratory Simulation

- 1) An optical system consists of two thick lenses (medium around is air), the first lens has refractive index $n_1 = 1.50$, thickness: $t_1 = 5mm$ surface radii

$$R_1 = +40mm$$

$$R_2 = -40mm$$

And lens 2 is a distance $d_{12} = 30mm$ (distance vertex to vertex), refractive index: $n_2 = 1.6$, thickness: $t_2 = 4mm$, surface radii

$$R_3 = +25mm$$

$$R_4 = -35mm$$

1.1 Computing values

For calculating the focal points, we find the following:

$$\frac{1}{f} = (n_l - 1) \left(\frac{1}{R_1} - \frac{1}{R_2} + \frac{(n_l - 1)t}{n_l R_1 R_2} \right)$$

$$\Rightarrow f_1 = \boxed{+40.85mm}$$

$$\Rightarrow f_2 = \boxed{+24.93mm}$$

For calculating the principle planes we use the following formula:

$$h_n = -\frac{f(n-1)t}{nR_n}$$

$$\Rightarrow h_{11} = \boxed{-1.702mm}$$

$$\Rightarrow h_{12} = \boxed{1.702mm}$$

$$\Rightarrow h_{21} = \boxed{-1.496mm}$$

$$\Rightarrow h_{22} = \boxed{1.068mm}$$

1.2 Using Matrix Formalism

For a **thick lens in air**, the system matrix is:

$$M = \begin{bmatrix} 1 & 0 \\ \frac{n_l-1}{R_2} & \frac{n_l}{1} \end{bmatrix} \begin{bmatrix} 1 & t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ \frac{1-n_l}{R_1 n_l} & \frac{1}{n_l} \end{bmatrix}$$

1.3 Translation between lenses

Your translation matrix is correct:

$$T_{12} = \begin{bmatrix} 1 & d_{12} \\ 0 & 1 \end{bmatrix}$$

```
[9]: import numpy as np
import matplotlib.pyplot as plt

# --- Optical elements ---

def refraction_matrix(n1, n2, R):
    return np.array([
        [1, 0],
        [(n1 - n2) / (R * n2), n1 / n2]
    ])

def translation_matrix(d):
    return np.array([
        [1, d],
        [0, 1]
    ])

def thick_lens_matrix(n, R1, R2, t):
    # air -> lens
    refraction1 = refraction_matrix(1.0, n, R1)
    # propagation inside lens
    translation = translation_matrix(t)
    # lens -> air
    refraction2 = refraction_matrix(n, 1.0, R2)

    return refraction2 @ translation @ refraction1

# --- Ray tracing ---
def run_simulation(system, system_xvalues, incoming):
```

```

coords = []

for ray0 in incoming:
    ray = ray0.copy()
    ray_coords = []

    for matrix in system:
        rayf = matrix @ ray
        ray_coords.append((ray.copy(), rayf.copy()))
        ray = rayf

    coords.append(ray_coords)

# --- Plotting (height vs x) ---

for ray_coords in coords:
    for i in range(len(ray_coords)):
        y_start = ray_coords[i][0][0]
        y_end = ray_coords[i][1][0]
        x_vals = system_xvalues[i]
        y_vals = [y_start, y_end]

        plt.plot(x_vals, y_vals)

plt.xlabel("Optical axis (x)")
plt.ylabel("Ray height (y)")
plt.title("Paraxial Ray Trace")
plt.grid(True)
plt.show()

# --- Total system matrix ---

M = np.array([[1,0],[0,1]])
for matrix in system:
    M = matrix @ M
print("System matrix:\n", M)

A, B = M[0]
C, D = M[1]

f_eff = -1 / C

print("Effective focal length:", f_eff, "mm")

# --- System definition ---

```

```

air1 = translation_matrix(100)
lens1 = thick_lens_matrix(1.5, 40, -40, 5)
air2 = translation_matrix(30)
lens2 = thick_lens_matrix(1.6, 25, -35, 4)
air3 = translation_matrix(100)

system = [air1, lens1, air2, lens2, air3]

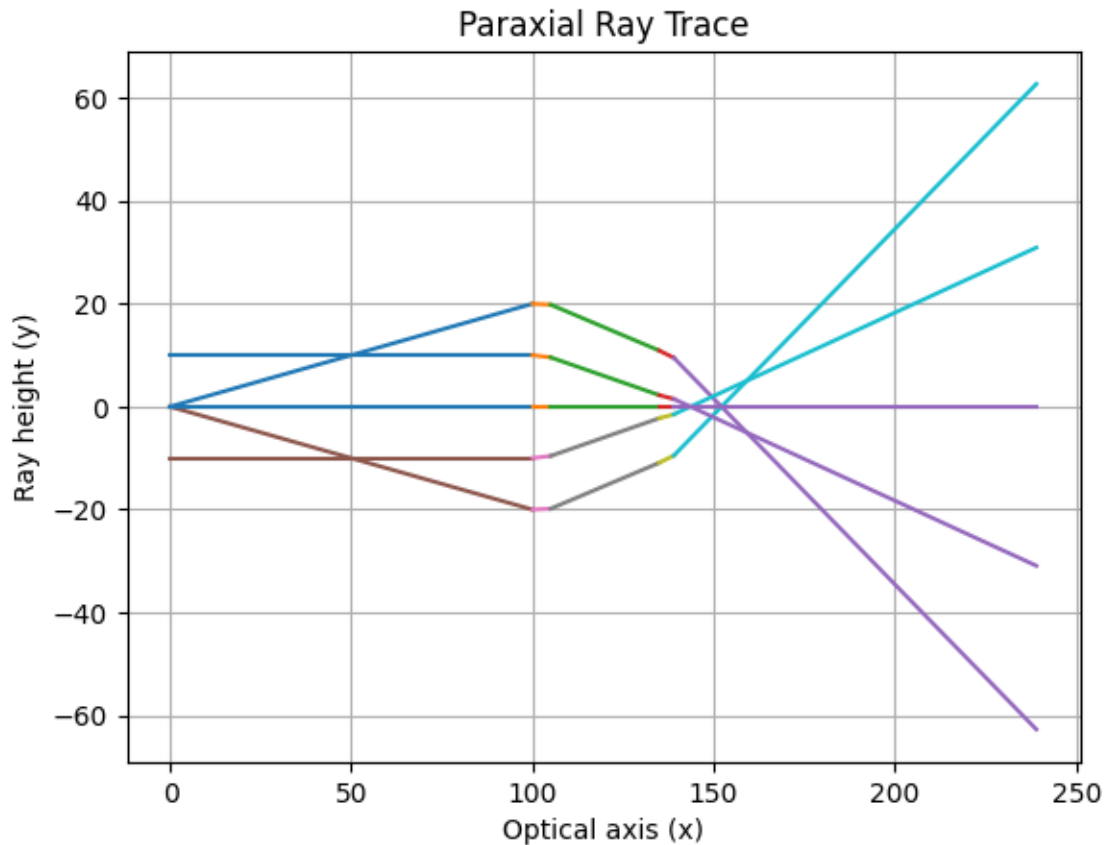
# x-positions for plotting each segment
system_xvalues = [
    [0, 100],
    [100, 105],
    [105, 135],
    [135, 139],
    [139, 239]
]

# --- Incoming rays (y, theta) ---

incoming = [
    np.array([0.0, 0.2]),
    np.array([0.0, -0.2]),
    np.array([10.0, 0.0]),
    np.array([-10.0, 0.0]),
    np.array([0.0, 0.0])
]

run_simulation(system, system_xvalues, incoming)

```



System matrix:

```
[[-3.09207589e+00 -3.13627232e+02]
```

```
[-3.24139881e-02 -3.61113690e+00]]
```

Effective focal length: 30.850878240030845 mm

```
[10]: # --- Define a new system ---

air3 = translation_matrix(20)
lens3 = thick_lens_matrix(1.55, 50, -50, 6)
air4 = translation_matrix(100)

system = [air1, lens1, air2, lens2, air3, lens3, air4]

# --- New x values for plotting ---

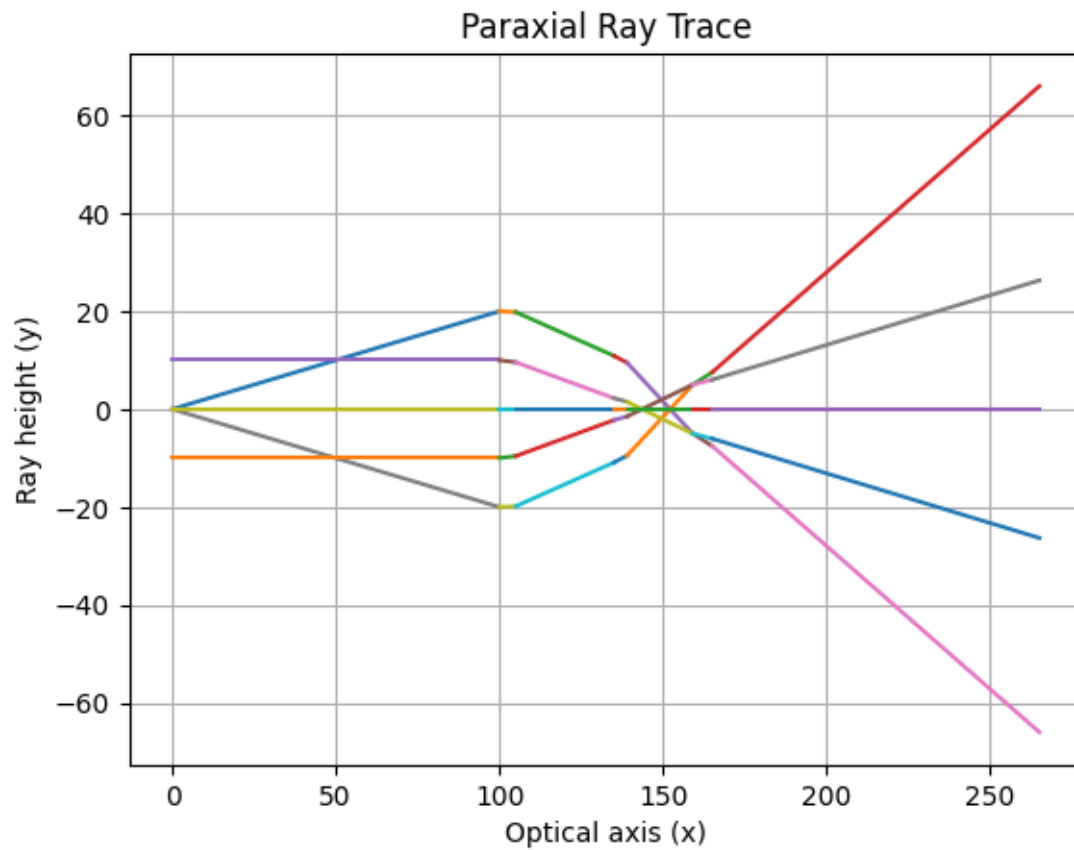
system_xvalues = [
    [0, 100],
    [100, 105],
    [105, 135],
    [135, 139],
```

```

[139, 159],
[159, 165],
[165, 265]
]

run_simulation(system, system_xvalues, incoming)

```



System matrix:

```

[[-2.63222784e+00 -3.30137624e+02]
 [-2.02904339e-02 -2.92476037e+00]]
Effective focal length: 49.28430825108046 mm

```

[]: